

Каталог из 84 численных форматов с битоточными векторами соответствия:

вендор-нейтральный справочник для FP8, BF16, MXFP4 и
микромасштабируемых форматов

Дмитрий Васильев (Dmitrii Vasilev)

Независимый исследователь, Trinity S³AI

Контакт: dao999nft@gmail.com · GitHub: github.com/gHashTag

ORCID: [0009-0008-4294-6159](https://orcid.org/0009-0008-4294-6159)

Препринт v4 · 8 июня 2026 г.

АННОТАЦИЯ

Распространение численных форматов в аппаратном обеспечении для машинного обучения — FP8 (E4M3 и E5M2), BF16, MXFP4, блочные микромасштабируемые форматы и десятки исследовательских вариантов — опередило доступность вендор-нейтральных битоточных справочных материалов. Инженеры, переносящие модели между ускорителями, сталкиваются с незаметными расхождениями, которые трудно диагностировать без общей измерительной линейки.

В настоящей статье описаны: каталог из 84 численных форматов, охватывающий 13 семейств; набор из шести битоточных пакетов соответствия (conformance packs) для GF16, элемента MXFP4, BF16, FP8 E4M3, FP8 E5M2 и блочного масштаба E8M0; а также кросс-уолк к IEEE P3109 v3.2.0, сопоставляющий каждый пакет с соответствующей конфигурируемой формой стандарта. Каждый пакет представляет собой самодостаточный документ JSON с отпечатком SHA-256, общей схемой строк и якорным вектором, кодирующим значение $3,0$ — тождество $\varphi^2 + 1/\varphi^2 = 3$ — в качестве межпакетной проверки корректности. Пакеты перекрёстно проверяются относительно ml_dtypes 0.5.4 (Google/JAX); любое расхождение документируется явно и интерпретируется как разрешённый спецификацией интерпретационный разрыв, а не скрывается. Работа позиционируется как заполнение реестра: она не предлагает новых форматов, не делает утверждений о точности моделей и не заявляет о превосходстве над какой-либо вендорской реализацией. Все артефакты публично доступны по адресу github.com/gHashTag/t27 под открытой лицензией.

Ключевые слова: численные форматы, форматы с плавающей точкой, conformance, IEEE P3109, золотое сечение, машинное обучение, битоточные векторы, микромасштабирование, FP8, BF16, MXFP4.

1. Введение

Представьте себе механика, которому нужно подогнать деталь под спецификацию токарного станка, но линейка в его руках использует единицы, слегка отличающиеся от тех, что указаны на чертеже. Деталь может выглядеть правильной; расхождение проявляется только под нагрузкой. Тот же сценарий разыгрывается в прошивках ML-ускорителей: два чипа могут одновременно заявлять о «поддержке FP8 E4M3», однако незаметно различаться в обработке случая переполнения для входного значения вроде 1000,0 — один насыщается до максимального конечного значения 448,0, другой переходит в NaN. Спецификация OCP Microscaling допускает оба варианта. Без общего битоточного эталона перенесённая модель может выдавать численно различающиеся результаты, которые трудно локализовать.

В настоящей статье описаны два артефакта, призванные служить такой общей линейкой.

Вклад 1: каталог из 84 численных форматов. Каталог t27 перечисляет 84 численных формата в 13 семействах (раздел 3). Каждая запись несёт единообразную схему: разрядную раскладку, смещение порядка (bias), политику бесконечности/NaN, политику насыщения, максимальное конечное значение, минимальное нормальное значение, минимальное субнормальное число и метку статуса утверждения (Verified / Empirical_fit / Open_conjecture / Risk / Retracted). Каталог хранится как единый источник истины и кросс-компилируется в Markdown, JSON, Python, Rust, C и TypeScript посредством шаблонного инструмента.

Вклад 2: шесть битоточных пакетов соответствия. Пакеты (раздел 5) охватывают шесть форматов, наиболее часто встречающихся в современном промышленном аппаратном обеспечении и исследовательских конвейерах: GoldenFloat 16 (GF16), элемент MXFP4, BF16, FP8 E4M3, FP8 E5M2 и блочный масштаб E8M0. Два пакета (GF16 и MXFP4) уже доступны в PyPI-пакете tt-lang-t27 v0.3.1; остальные четыре представлены в текущем предварительном выпуске, доступном по адресу github.com/gHашTag/tt-lang-t27/pull/6.

Чем эта статья не является. В работе не приводятся бенчмарки точности моделей, не предлагаются новые форматы и не проводятся сравнения производительности между вендорами. Читателям, которым нужен анализ пропускной способности FLOP или результаты по точности квантования, следует обратиться к отдельной литературе.

Структура статьи. Раздел 2 рассматривает соответствующий ландшафт стандартов. Раздел 3 описывает устройство каталога. Раздел 4 определяет методологию пакетов соответствия. Раздел 5 поочерёдно представляет каждый из шести пакетов. Раздел 6 предоставляет кросс-уолк к IEEE P3109. Раздел 7 обсуждает интерпретационный разрыв как проектную особенность. Раздел 8 охватывает воспроизводимость и происхождение. Раздел 9 намечает будущую работу. Раздел 10 формулирует ограничения.

Историческая преемственность. Один из шести пакетов соответствия — GoldenFloat 16 (GF16) — относится к отечественной линии исследований оригинальных систем

представления чисел, начатой троичной ЭВМ «Сетунь» под руководством Н. П. Брусенцова (Вычислительный центр МГУ, 1958) — первой в мире машиной на симметричном троичном коде $(-1, 0, +1)$, а в «Сетунь-70» (1970) — с введённым троичным форматом «трайт» и оптимизированным диапазоном мантиссы нормализованного числа. По доступным открытым источникам, GF16 — один из первых именованных числовых форматов российского авторства, доведённых до уровня публикации и аппаратной реализации (FPGA).

2. Предпосылки и предшествующие работы

Распространение низкоточных форматов с плавающей точкой в аппаратном обеспечении для машинного обучения было охарактеризовано — ещё в популярном эссе 2017 года — как «дикий запад компьютерной арифметики». Почти десятилетие спустя ландшафт стал богаче (больше вендоров, больше блочных форматов, больше исследовательских вариантов), однако вендор-нейтральные битоточные справочные материалы не поспевают за ним.

2.1. Стандарты с плавающей точкой

IEEE 754-2019 определяет двоичные форматы обмена (binary16, binary32, binary64, binary128) и правила округления, переполнения и обработки NaN, которые ими управляют. BF16 (brain float 16) отсутствует в редакции 2019 года; он возник неформально в Google Brain и теперь поддерживается аппаратурой Intel, AMD, ARM и NVIDIA, разделяя диапазон порядка FP32 (8 бит порядка, смещение порядка = 127) с 7-битной мантиссой.

Форматы FP8 прошли схожую траекторию «сначала неформально, затем формально». Нун (Noune) и др. в 2022 году провели обзор 8-битных численных форматов для глубоких нейронных сетей, мотивировав варианты E4M3 и E5M2, впоследствии принятые в OCP MX и IEEE P3109.

Спецификация OCP Microscaling (MX) v1.0 вводит блочные форматы, в которых группы из 32 элементов разделяют общий масштабный множитель E8M0. Типы элементов включают MXFP4 (S1E2M1), MXFP6 (E2M3 и E3M2), MXFP8 (E4M3 и E5M2) и MXINT8. OCP MX явно допускает две политики переполнения для FP8 E4M3: насыщение до максимального конечного значения (используется в реализациях tt-metal и AMD) и переполнение в NaN (используется в JAX/TPU).

NVFP4 от NVIDIA — недавний 4-битный вариант, сочетающий элемент в стиле MXFP4 (S1E2M1) с 16-элементным блоком (меньшим, чем 32-элементный блок OCP MX) и использующий блочный масштаб FP8 E4M3 вместо E8M0. На момент написания NVFP4 задокументирован в форме технического блога NVIDIA и программных стеков Blackwell/Rubin; он пока не охвачен открытой межвендорной спецификацией. M²XFP далее обобщает микромасштабирование, допуская смешанные точности элементов внутри блока, причём аппаратная реализуемость исследуется в горизонте ASPLOS '26.

IEEE P3109 — активная рабочая группа, стандартизирующая 8-битные и 4-битные форматы с плавающей точкой для задач ИИ. Её промежуточный отчёт v3.2.0 определяет Binary8p3se и Binary4p1sf (среди прочих) и каталог StandardOperations.yaml из приблизительно 80 операций в семи категориях.

2.2. Существующие эталонные реализации

ml_dtypes (Google/JAX) — библиотека Python/C++, предоставляющая эталонные реализации bfloat16, float8_e4m3fn, float8_e5m2, float8_e8m0fnu и нескольких других форматов. Она служит эталонным оракулом на протяжении всей настоящей работы.

P3109 FLoPS — формализация семантики P3109 на Lean 4, обеспечивающая доказательно проверенное покрытие ключевых операций.

Pyshop (Carson & Chen, 2025) эмулирует широкое семейство низкоточных арифметик в Python, включая варианты FP8, posit и настраиваемые кортежи (E, M, смещение порядка). Она ориентирована на задачи ML и научных вычислений и дополняет настоящий каталог: Pyshop эмулирует поведение на уровне операций, тогда как пакеты в этой статье фиксируют битовые паттерны на уровне представления.

libtakum — эталонная C-библиотека для семейства арифметики takum (Хунхольд), обеспечивающая пригодную базу для кросс-реализации кластера Posit/Unum III каталога относительно независимого оракула.

torch.float8 (PyTorch) и **jax.dtypes** предоставляют типы FP8 на уровне фреймворка, но не публикуют наборы тестов с битовыми векторами, независимые от аппаратного исполнения.

Исследования по оценке MX измеряют влияние микромасштабируемого квантования на точность в трансформерных задачах.

2.3. Разрыв

Ни один отдельный вендор-нейтральный артефакт в настоящее время не охватывает FP8 E4M3, FP8 E5M2, BF16, элемент MXFP4, элемент NVFP4, GoldenFloat 16 и блочный масштаб E8M0 в единой схеме, обладая при этом: (а) битоточными векторами кодирования/декодирования; (б) происхождением, привязанным к SHA-256; (в) явной документацией каждого расхождения с эталонной реализацией; и (г) удобочитаемым кросс-уолком к IEEE P3109. Настоящая работа заполняет этот пробел реестра для шести наиболее немедленно развёртываемых пакетов уровня представления; NVFP4 обсуждается как ближайший кандидат «Трека 2» (раздел 9) и как второй интерпретационный разрыв (раздел 7).

3. Устройство каталога

3.1. 84 формата в 13 кластерах

Каталог t27 содержит 84 формата, организованных в 13 именованных кластеров. Таблица 1 показывает названия кластеров и количество форматов. Сумма количеств равна ровно 84; это непрерывно контролируемый инвариант каталога (CI-01, раздел 3.4).

Кластер	Представительные форматы	Кол-во
IEEE754 binary	binary16, binary32, binary64, binary128, binary80	5
IEEE754 decimal	decimal32, decimal64, decimal128	3
MLLowPrecision	BF16, TF32, FP8 E4M3, FP8 E5M2, FP8 E3M4, FP6, FP4, NF4	8
GoldenFloat	GF4 ... GF256 (варианты с привязкой к ф)	16
Posit / Unum III	Posit8, Posit16, Posit32, takum8, takum16, ...	8
OCP MX	MXFP4, MXFP6_E2M3, MXFP6_E3M2, MXFP8_E4M3, E8M0_block	5
LNS	варианты LNS8, LNS16	4
IntegerFixed	INT2, INT4, INT8, INT16, UINT4, UINT8, UINT16, FXP16	8
HistoricalVendor	IBM hex float, DEC VAX G, Cray single, NVIDIA TF32, ...	10
Theoretical	граничные случаи E0M7, E7M0, E1M6, E6M1	4
Compression/scaling	NF4 block, E8M0 block, SF8, RFP8	4
Extended	binary256, bfloat32, bfloat128	3
QuantTuned	Q-BF16, adaptive-FP8	2
Итого		84

Таблица 1. 84 формата в 13 кластерах (T1).

3.2. Схема «одна строка на формат»

Каждая запись каталога несёт следующие поля:

- name — канонический идентификатор (ASCII, без пробелов)
- bits — общая разрядность
- exp — ширина поля порядка в битах
- mant — ширина поля мантиссы в битах (0 для форматов в стиле E8M0)
- bias — смещение порядка
- has_inf — булево значение
- has_nan — булево значение
- saturation_policy — SatFinite, OvfInf или OvfNaN

- `max_finite` — наибольшее представимое конечное значение (f64)
- `min_normal` — наименьшее положительное нормальное значение (f64)
- `min_subnormal` — наименьшее положительное субнормальное число (f64; null, если отсутствует)
- `cluster` — одна из 13 меток кластеров
- `claim_status` — Verified / Empirical_fit / Open_conjecture / Risk / Retracted

3.3. Таксономия статусов утверждений

Verified: спецификация формата подкреплена опубликованным стандартом (IEEE, OCP) или доказательно проверенным эталоном (P3109 FLoPS Lean).

Empirical_fit: выведена путём подгонки наблюдаемой битовой раскладки аппаратного продукта без независимо опубликованной спецификации.

Open_conjecture: предложенное обобщение, ожидающее внешней проверки.

Risk: ссылка на спецификацию существует, но кодировка в каталоге может содержать ошибки, ещё не выявленные набором тестов.

Retracted: ранее была включена; удалена после выявления противоречащего авторитетного источника.

3.4. Инварианты каталога

Пятнадцать инвариантов проверяются при каждом коммите. Отдельные инварианты перечислены в таблице 2.

ID	Инвариант	Проверка
CI-01	Общее число форматов равно 84	<code>sum(cluster_counts) == 84</code>
CI-02	Нет коллизий имён	<code>len(names) == len(set(names))</code>
CI-03	Согласованность разрядности	<code>1 + exp + mant == bits</code> для стандартной раскладки
CI-04	Диапазон смещения порядка	<code>bias <= 2**(exp-1) - 1</code>
CI-05	Наличие политики насыщения	поле не равно null для каждой записи
CI-06	Положительное max-finite	<code>max_finite > 0</code>
CI-07	$\text{min-normal} \leq \text{max-finite}$	порядок сохранён
CI-08	Метка кластера из перечисления	нет неперечисленных имён кластеров
CI-09	Статус утверждения из перечисления	нет неперечисленных значений статуса
CI-10	Нет формата с 0 бит	<code>bits >= 2</code>
CI-11	Наличие якоря SHA-256	заголовок каждого пакета несёт поле отпечатка
CI-12	Наличие якорного вектора	каждый пакет содержит вектор <code>anchor_*</code>

ID	Инвариант	Проверка
CI-13	Якорь декодируется в 3,0	<code>decode(anchor_bits) == 3.0</code>
CI-14	Цели кодогенерации компилируются	CI-матрица запускает тесты импорта Python + Rust
CI-15	Нет дублирующихся SHA-256	отпечатки пакетов глобально уникальны

Таблица 2. 15 инвариантов каталога (контролируемые CI) (T2).

3.5. Путь кодогенерации

Единый шаблонный инструмент на Jinja2 читает канонический JSON-каталог и порождает выходные файлы для каждого языка: Markdown (удобочитаемая таблица), JSON (экспорт для API), классы данных Python, структуры Rust с производными `serde`, заголовок C (константы `#define`) и литералы перечислений TypeScript. Все сгенерированные файлы фиксируются в репозитории github.com/gHashTag/t27 и пересобираются при каждом push посредством матрицы GitHub Actions.

4. Методология пакетов соответствия

4.1. Общая схема строк

Каждый пакет соответствия представляет собой JSON-массив векторов, причём каждая строка соответствует схеме, показанной в таблице 3.

Поле	Тип	Описание
name	string	удобочитаемый идентификатор тест-случая
input_f64	number	входное значение как число двойной точности
input_f64_hex	string	шестнадцатеричная кодировка IEEE 754 входа (f32 или f64)
<fmt>_bits_hex	string	битовый паттерн целевого формата, hex
<fmt>_bits_int	integer	тот же битовый паттерн как беззнаковое целое
decoded_f64	number	результат <code>decode(encode(input))</code>
decoded_f64_hex	string	шестнадцатеричная кодировка IEEE 754 декодированного значения
abs_error	number	вход – декодированное ; всегда показывается (никогда не скрывается)
category	string	zero / normal / subnormal / inf / nan / overflow / underflow / rounding / anchor / transcendental

Таблица 3. Общая схема строк для всех пакетов соответствия (T3).

4.2. Заголовок пакета

Помимо массива векторов, каждый файл пакета несёт объект-заголовок со следующими полями:

- Четвёрка спецификации формата: (E, M, смещение порядка, политика inf/NaN)
- Политика насыщения
- Максимальное конечное значение

- Самоотпечаток SHA-256 (вычисленный по канонической JSON-сериализации)
- Якорь версии `ml_dtypes`
- Ссылка на якорное тождество: $\phi^2 + 1/\phi^2 = 3$ (arXiv:2606.05017)

4.3. Якорный вектор

Каждый пакет содержит как минимум один вектор с именем `anchor_*`, кодирующий значение 3,0. Мотивацией служит тождество

$$\phi^2 + 1/\phi^2 = 3, \quad (1)$$

где $\phi = (1 + \sqrt{5})/2$ — золотое сечение. Это тождество представлено и контекстуализировано в препринте `GoldenFloat` как численно обоснованный L_2 -якорь. Значение 3,0 точно представимо во всех шести форматах пакетов (оно попадает в нормальный диапазон с нулевой ошибкой мантиссы для всех шести раскладок), что делает его надёжной однострочной проверкой корректности между пакетами.

Формально: для любого формата пакета F , если $\text{decode}_F(\text{encode}_F(3,0)) \neq 3,0$, присутствует фундаментальная ошибка реализации.

4.4. Шаги верификации

Каждый пакет проверяется двумя независимыми процедурами:

1. **Самопроверка циклическим кодированием-декодированием (round-trip).** Для каждого вектора: $\text{decode}(\text{encode}(\text{input})) = \text{decoded}$, причём сохранённое значение `abs_error` согласовано с отклонением.
2. **Перекрёстная проверка относительно `ml_dtypes 0.5.4`.** Там, где существует соответствующий тип `ml_dtypes`, битовые паттерны пакета сравниваются с кодировкой `ml_dtypes` тех же входов. Каждое расхождение записывается в список `divergences` заголовка пакета и описывается в настоящей статье.

Честная трактовка абсолютной ошибки — непреложный принцип проектирования. Каждый вектор, в котором декодированное значение отличается от входного, несёт ненулевое значение `abs_error`; никакое значение не подавляется и не округляется до нуля ради улучшения статистики совпадений.

5. Шесть пакетов соответствия

Таблица 4 даёт сводку по всем шести пакетам.

Пакет	Раскладка	Век-ры	Совпадение <code>ml_dtypes</code>	Статус	SHA-256 (16)
GF16	S1E5M10, привязка к ϕ	21	н/п (нет аналога в <code>ml_dtypes</code>)	LIVE v0.3.1	см. репозиторий
MXFP4	S1E2M1, элемент блока	12	н/п	LIVE v0.3.1	86c99d6f72375d75

Пакет	Раскладка	Век-ры	Совпадение ml_dtypes	Статус	SHA-256 (16)
BF16	S1E8M7 bias=127, RTE	21	21/21	NEW v0.4.0-pre	320c1850b4846745
FP8 E4M3	S1E4M3 bias=7, SatMax	16	15/16 (раздел 7)	NEW v0.4.0-pre	fff0c30f8e6bee22
FP8 E5M2	S1E5M2 bias=15, Ovflnf	17	17/17	NEW v0.4.0-pre	66cd7be1500ec800
E8M0 block	E8M0 только масштаб, без знака	11	согласован с OCP MX v1.0	NEW v0.4.0-pre	b211f1a863f71fd7

Таблица 4. Шесть пакетов с первого взгляда (T4).

Полные отпечатки SHA-256 (дословно из манифеста):

- **GF16:** см. репозиторий (SHA-256 пока не закреплён в манифесте v0.4.0-pre)
- **MXFP4:** 86c99d6f72375d751df4c74897904a0a36cff52e8d60cbfef5d58b71625d4b2f
- **BF16:** 320c1850b484674546785791b1c22d76feb4ea748c6669ffb633e5455d822b8a
- **FP8 E4M3:** fff0c30f8e6bee22b1a7d0e0e1cff65edde9d2b17ebf97dba0539973f0a5e89d
- **FP8 E5M2:** 66cd7be1500ec8003eb5dee7532bb4e954b7bc0084b6f22a75d02f7842f23a56
- **E8M0 block:** b211f1a863f71fd7c5e02e512efff0255ebcc51521311186e01cb9992e4464bd

5.1. GF16 — GoldenFloat 16-бит

GF16 — это 16-битный формат с раскладкой S1E5M10 и ф-поворотом представимого диапазона. Он описан и обоснован в препринте GoldenFloat. Пакет содержит 21 вектор, охватывающий нуль, нормальные значения, якорь 3,0 (кодирующий тождество $\varphi^2 + 1/\varphi^2 = 3$, ур. (1)), субнормальные числа и поведение при переполнении. Поскольку ml_dtypes не реализует тип GF16, у этого пакета нет партнёра для перекрёстной проверки; его векторы верифицируются только самопроверкой циклическим кодированием-декодированием. GF16 доступен в PyPI-пакете tt-lang-t27 начиная с версии v0.3.1.

5.2. Элемент MXFP4 — OCP Microscaling 4-бит

Элемент MXFP4 использует раскладку S1E2M1 (1 бит знака, 2 бита порядка, 1 бит мантиссы) с политикой переполнения «насыщение до конечного», как указано в OCP MX v1.0. В пределах блока 32 таких элемента разделяют масштабный множитель E8M0. Пакет элемента охватывает 12 векторов: 15 представимых конечных значений плюс нуль и случай насыщения. На момент написания ml_dtypes не предоставляет тип элемента MXFP4; пакет верифицируется самопроверкой циклическим кодированием-декодированием и сравнивается с таблицей значений OCP MX v1.0. SHA-256: 86c99d6f72375d751df4c74897904a0a36cff52e8d60cbfef5d58b71625d4b2f.

5.3. BF16 — Brain Float 16

BF16 использует раскладку S1E8M7 со смещением порядка = 127, округлением к ближайшему чётному (RTE) и обработкой бесконечности и NaN в стиле IEEE 754. Он занимает старшие 16 бит FP32-слова, так что преобразование в/из FP32 — это простое усечение (с округлением). Пакет содержит 21 вектор, включая:

- Положительный и отрицательный нуль
- Положительную и отрицательную бесконечность (сохраняются точно)
- Тихий NaN (сохраняется с полезной нагрузкой)
- Наименьшее положительное нормальное и субнормальное
- Наибольшее конечное BF16 ($\approx 3,39 \times 10^{38}$)
- Два случая середины RTE (поведение округления к чётному)
- Переполнение FP32-максимума в BF16 $+\infty$ (`abs_error =  $+\infty$`)
- Потеря значимости FP32-минимума-субнормали в BF16 $+0$
- Неточные константы ϕ и $1/\phi$ с ненулевой `abs_error`
- Якорный вектор при 3,0 (точно, `abs_error = 0`)

Все 21 вектор совпадают с `ml_dtypes.bfloat16` (Google/JAX 0.5.4): 21/21. SHA-256: 320c1850b484674546785791b1c22d76feb4ea748c6669ffb633e5455d822b8a.

BF16 демонстрирует высокое межвендорное согласие; Google `bfloat16`, Intel `BFLOAT16`, ARM `BFloat16` и BF16, сопряжённый с NVIDIA TF32, разделяют одну и ту же семантику `sub/inf/NaN` в стиле IEEE 754 с округлением к ближайшему чётному на младших 16 битах FP32. В 21 протестированном граничном случае заметных расхождений не наблюдалось.

5.4. FP8 E4M3 — восьмибитный float с четырёхбитным порядком

FP8 E4M3 использует раскладку S1E4M3 со смещением порядка = 7. В варианте OCP MX (используемом здесь) бесконечность заменена дополнительными конечными значениями, а NaN кодируется битовым паттерном 0x7F (или 0xFF для отрицательных). Таким образом, формат не имеет $+\infty$, что даёт максимальное конечное значение 448,0.

Пакет содержит 16 векторов. 15 из 16 точно совпадают с `ml_dtypes.float8_e4m3fn`. Единственное задокументированное расхождение — случай переполнения для входа 1000,0, подробно описанный в таблице 5 и обсуждаемый в разделе 7. SHA-256: fff0c30f8e6bee22b1a7d0e0e1cff65edde9d2b17ebf97dba0539973f0a5e89d.

Вход	Реализация	Биты	Декодировано	Политика
1000,0	этот пакет (конвенция tt-metal/AMD)	0x7E	448,0 (max-finite)	насыщение до максимума
1000,0	<code>ml_dtypes</code> 0.5.4 (конвенция JAX/TPU)	0x7F	NaN	переполнение в NaN

Оба варианта разрешены OCP MX v1.0. См. раздел 7.

Таблица 5. Интерпретационный разрыв при переполнении FP8 E4M3 для входа 1000,0 (T5).

5.5. FP8 E5M2 — восьмибитный float с пятибитным порядком

FP8 E5M2 использует раскладку S1E5M2 со смещением порядка = 15 и сохраняет полноценные бесконечность и NaN в стиле IEEE 754. Максимальное конечное значение — 57344,0. Пакет содержит 17 векторов, охватывающих полный набор граничных случаев (нуль, нормальные, субнормальные, $\pm\infty$, NaN, переполнение, потеря значимости, середины RTE и якорь 3,0). Все 17 векторов точно совпадают с `ml_dtypes.float8_e5m2`: 17/17. SHA-256: 66cd7be1500ec8003eb5dee7532bb4e954b7bc0084b6f22a75d02f7842f23a56.

5.6. Блочный масштаб E8M0 — масштабный формат OCP Microscaling

E8M0 — формат «только масштаб», используемый как общий блочный порядок в блоках OCP MX. Он не несёт ни бита знака, ни мантиссы — только 8 бит порядка, представляющих степени 2 в диапазоне $[2^{-127}, 2^{127}]$. Специальный паттерн 0xFF кодирует NaN (используется для обозначения неинициализированного или недопустимого масштаба). Пакет содержит 11 векторов, охватывающих представительные значения масштаба, сторожевое значение NaN и якорь 3,0 (который кодируется в ближайший представимый масштаб-степень-двойки, $2^1 = 2$, с задокументированной ненулевой `abs_error`). Векторы были регенерированы относительно `ml_dtypes.float8_e8m0fnu` (Google/JAX 0.5.4) по семантике OCP MX v1.0. SHA-256: b211f1a863f71fd7c5e02e512efff0255ebcc51521311186e01cb9992e4464bd.

6. Кросс-уолк к IEEE P3109

IEEE P3109 — активная рабочая группа, стандартизирующая арифметику с плавающей точкой для приложений ИИ. Её промежуточный отчёт v3.2.0 определяет семейство конфигурируемых форматов, параметризованных тройкой (E, M, насыщение). Более широкое обоснование явного побитового тестирования соответствия в промышленной практике с плавающей точкой приводит Винтерстайгер (Wintersteiger) на ARITH 2025; пакеты в этой статье — конкретный пример такого подхода, нацеленный именно на реестр численных форматов ИИ. Там, где существует машинно-проверяемая семантика, например разработка P3109 FLoPS на Lean 4, кросс-уолк в таблице 6 служит мостом между доказательно проверенной спецификацией и битоточными тестовыми данными.

Таблица 6 сопоставляет шесть пакетов с конфигурируемыми форматами P3109 v3.2.0.

Наш формат	Имя P3109	Совпадение	Ключевое различие	Пакет
FP8 E4M3	Binary8p3se	Близкое	Насыщение: у нас SatMax против OvInf в P3109; кодирование NaN только конечными	fp8_e4m3
MXFP4 элемент	Binary4p1sf	Прямое	Блочная структура ортогональна; элемент совпадает	mxfp4
GF16	(нет)	Нет совпадения	P3109 не охватывает 16-битные форматы с привязкой к ф	gf16

Наш формат	Имя P3109	Совпадение	Ключевое различие	Пакет
BF16	(нет)	Нет совпадения	P3109 фокусируется на 4/8-битных; BF16 — 16-битный	bf16
FP8 E5M2	(нет)	Нет совпадения	Binary8p2se соответствовал бы, но отсутствует в v3.2.0	fp8_e5m2
E8M0 block	(нет)	Ортогонально	P3109 не определяет формат «только масштаб»	e8m0_block

Таблица 6. Кросс-уолк P3109 v3.2.0 для шести пакетов (T6).

6.1. Прямые совпадения

Binary8p3se ↔ FP8 E4M3. P3109 Binary8p3se специфицирует S1E4M3 с насыщением OvfInf. Вариант FP8 E4M3 из OCP MX v1.0, используемый в этом пакете, применяет вместо этого SatMax. Различие — ровно тот интерпретационный разрыв при переполнении, что задокументирован в таблице 5. За исключением выбора политики насыщения, битовые раскладки и смещение порядка идентичны.

Binary4p1sf ↔ MXFP4 элемент. P3109 Binary4p1sf специфицирует S1E2M1 с SatFinite — идентично раскладке элемента MXFP4 в OCP MX v1.0. Единственное структурное различие в том, что OCP MX оборачивает элементы в 32-элементные блоки, разделяющие масштабный множитель E8M0 — блочное измерение, которое P3109 не рассматривает в v3.2.0.

6.2. Частичные совпадения и несовпадения

FP8 E5M2 отображался бы в гипотетический Binary8p2se, который отсутствует в профилях P3109 v3.2.0. GF16 и BF16 находятся вне 4/8-битной области, которую P3109 в настоящее время охватывает. E8M0 — формат «только масштаб», ортогональный уровню представления P3109.

6.3. Охват операций

Файл StandardOperations.yaml стандарта P3109 перечисляет приблизительно 80 операций в семи категориях: классификация (8), сравнение (7), экстремумы (10+), округление проекцией (6 режимов), арифметика (10), трансцендентные функции (≈25) и блочные операции (40+).

Текущий набор (v0.1) охватывает только уровень представления — битоточность кодирования/декодирования. Трек 2 (целевой срок III кв. 2026) расширит охват до уровня операций, как минимум округление NearestTiesToEven для сложения, умножения и FMA по всем шести форматам, с доказательно проверенной семантикой, взятой из P3109 FLoPS в качестве формального якоря.

7. Обсуждение: интерпретационный разрыв как ценность линейки

Набор тестов соответствия оправдывает себя не тогда, когда все векторы совпадают, а когда он вскрывает расхождение, которое иначе было бы невидимым. Два таких случая заслуживают явного упоминания: разрыв при переполнении FP8 E4M3 (раздел 7.1) и разрыв блочной структуры между MXFP4 и NVFP4 (раздел 7.2).

7.1. Разрыв А: политика переполнения FP8 E4M3

Случай переполнения FP8 E4M3 (вход = 1000,0) — канонический пример.

Спецификация OCP MX v1.0 утверждает, что для входов, превышающих максимальное конечное значение (448,0 для E4M3), реализации могут либо насыщаться до максимального конечного, либо выдавать NaN. Две зрелые реализации промышленного качества делают разный выбор:

- **Конвенция tt-metal (Tenstorrent) / AMD:** насыщение до максимального конечного. Битовый паттерн 0x7E, декодированное значение 448,0. Этот пакет принимает данную конвенцию.
- **Конвенция JAX/TPU (ml_dtypes 0.5.4):** переполнение в NaN. Битовый паттерн 0x7F декодированное значение NaN.

Ни один из вариантов не является ошибкой. Оба соответствуют OCP MX v1.0. Расхождение — задокументированный разрешённый спецификацией интерпретационный разрыв.

Практическое следствие значимо для авторов компиляторов и тестовых стендов. Любой межвендорный перенос вычисления FP8 E4M3 должен либо: (а) явно выбрать одну политику и задокументировать её, либо (б) нести оба вектора в своём эталонном наборе тестов, принимая, что входы из диапазона переполнения будут давать разные результаты на разном оборудовании.

Именно это и призван вскрывать пакет соответствия. Набор тестов, который сравнивает лишь «совпадают ли выходы на этом оборудовании?», никогда не увидел бы этого расхождения — обе реализации проходят собственные тесты. Общий битоточный эталон делает разрыв видимым.

7.2. Разрыв В: 4-битная блочная структура (MXFP4 против NVFP4)

Второй класс интерпретационного разрыва возникает уровнем выше — на уровне блочной структуры, а не битового паттерна элемента. OCP MX MXFP4 и NVIDIA NVFP4 разделяют одну и ту же раскладку элемента S1E2M1 (так что пакеты элементов битоидентичны на 4-битном уровне), однако оборачивают этот элемент в блоки разной формы с по-разному квантованными полями масштаба. Таблица 7 резюмирует расхождение параметров.

Параметр	OCP MX MXFP4	NVIDIA NVFP4
Раскладка элемента	S1E2M1 (4 бита)	S1E2M1 (4 бита)

Параметр	ОСР MX MXFP4	NVIDIA NVFP4
Биты мантиссы (элемент)	1	1
Размер блока	32 элемента	16 элементов
Формат масштаба	E8M0 (8-бит)	FP8 E4M3 (8-бит)
Биты порядка масштаба	8 (чистый порядок)	4
Биты мантиссы масштаба	0	3
Динамический диапазон масштаба	$2^{-127} \dots 2^{127}$	$\approx 2^{-9} \dots 448$
Гранулярность масштаба на декаду	1 (только степени двойки)	8 (3-битная мантисса)
Бит/элемент с учётом масштаба	$4 + 8/32 = 4,25$	$4 + 8/16 = 4,50$

Таблица 7. Параметры блочной структуры MXFP4 против NVFP4.

Из таблицы параметров вытекают три структурных следствия:

- Различное разрешение масштаба.** E8M0 (MXFP4) квантует блочный масштаб до степени двойки; масштаб FP8 E4M3 в NVFP4 предлагает $2^3 = 8$ кодов мантиссы на двоичный порядок и потому разрешает внутриблочный динамический диапазон в восемь раз тоньше в пределах своего представимого диапазона.
- Различный диапазон масштаба.** E8M0 охватывает приблизительно 2^{254} двоичных порядков (с учётом зарезервированных кодов); FP8 E4M3 насыщается при 448 и теряет значимость ниже $\approx 2^{-9}$. Тензор, чей поблочный масштаб естественно лежит вне диапазона FP8, представим в MXFP4, но не в NVFP4 без перемасштабирования на более высоком уровне.
- Различный эффективный бюджет битов.** Поэлементное хранение составляет 4,25 бита для MXFP4 (32-элементный блок) против 4,50 бита для NVFP4 (16-элементный блок). Два формата не сравнимы напрямую по битам; любое сравнение коэффициентов сжатия должно учитывать эту разницу накладных расходов в 5,9 % у NVFP4.

Поэтому тензор, сохранённый как MXFP4, и тот же тензор, сохранённый как NVFP4, могут совпадать по каждому битовому паттерну элемента, но различаться по декодированному значению, поскольку поблочный масштаб, на который они умножаются, использует другой (и по-другому квантованный) формат масштаба. Это структурный интерпретационный разрыв, а не разрыв значений: битоточность элемента не влечёт равенства декодированного тензора.

Показание линейки симметрично разрыву А. И MXFP4, и NVFP4 — соответствующие реализации разумно спроектированного 4-битного блочного формата; они просто делают разный выбор блочной структуры. Межвендорное развёртывание 4-битных весов требует явного объявления используемого блочного формата (размер блока + формат масштаба), а не только раскладки элемента. Объявлений только об элементе («модель использует 4-битные веса в форме MXFP4») недостаточно.

Настоящий каталог охватывает сторону MXFP4 этой пары как пакет уровня Tier-1 и перечисляет NVFP4 как ближайшего кандидата Трека 2 (раздел 9.1). Документирование структурного разрыва — первый шаг; парный пакет NVFP4 с совпадающей схемой и намеренным межпакетным вектором расхождения на представительной границе квантования масштаба закроет его.

7.3. Общий паттерн: разрешённый спецификацией выбор как показание линейки

Норма честности: каждый вектор в каждом пакете, где декодированное значение отличается от входного, несёт ненулевое значение `abs_error`. Переполнение в $\pm\infty$ показывает `abs_error = Inf`; потеря значимости до нуля показывает фактическую величину утраченного значения. Никакое поле `abs_error` не подавляется и не округляется до нуля ради улучшения статистики совпадений.

8. Воспроизводимость и происхождение

8.1. Исходные репозитории

- **gHashTag/t27** (github.com/gHashTag/t27): единый источник истины (SSOT) для каталога и пакетов соответствия уровня Tier-1. Здесь располагаются все JSON-файлы каталога, файлы пакетов, проверки инвариантов и шаблоны кодогенерации.
- **gHashTag/tt-lang-t27** (github.com/gHashTag/tt-lang-t27): зеркало на PyPI. Версия 0.3.1 доступна на PyPI (включает пакеты GF16 и MXFP4). Версия 0.4.0-pre, добавляющая четыре новых пакета, описанных в этой статье, доступна в PR #6.
- **gHashTag/tt-trinity-corona** (github.com/gHashTag/tt-trinity-corona): контекст кремниевого оракула уровня Tier-2 для постсиликонного аудита на GF180MCU; упоминается здесь одной строкой для полноты.

8.2. Эталонный инструмент

Основной оракул для всей перекрёстной проверки — `ml_dtypes` 0.5.4 (Google/JAX), доступный по адресу github.com/jax-ml/ml_dtypes. Конкретные используемые типы: `ml_dtypes.bfloat16`, `ml_dtypes.float8_e4m3fn`, `ml_dtypes.float8_e5m2` и `ml_dtypes.float8_e8m0fnu`.

8.3. Отпечаток якоря

Якорное тождество $\varphi^2 + 1/\varphi^2 = 3$ (ур. (1)), формализованное в препринте GoldenFloat, имеет следующий канонический отпечаток SHA-256:

218403e344779c890f302ad2c70af21fb765060dd794d793c7eacc1ef8f80e6b

Этот отпечаток покрывает каноническую UTF-8-кодировку строки тождества и служит внеполосной проверкой того, что цитируется правильная якорная статья.

8.4. Таблица происхождения пакетов

Таблица 8 перечисляет путь в репозитории, ветку/PR и полный SHA-256 для каждого пакета.

Пакет	Репозиторий	Ветка / PR	Полный SHA-256
GF16	gHashTag/t27	main (v0.3.1)	см. репозиторий (не закреплён в манифесте v0.4.0-pre)
MXFP4	gHashTag/t27	main (v0.3.1)	86c99d6f72375d751df4c74897904a0a36cff52e8d60cbfef5d58b71625d4b2f
BF16	gHashTag/t27	PR #6 (v0.4.0-pre)	320c1850b484674546785791b1c22d76feb4ea748c6669ffb633e5455d822b8a
FP8 E4M3	gHashTag/t27	PR #6 (v0.4.0-pre)	fff0c30f8e6bee22b1a7d0e0e1cff65edde9d2b17ebf97dba0539973f0a5e89d
FP8 E5M2	gHashTag/t27	PR #6 (v0.4.0-pre)	66cd7be1500ec8003eb5dee7532bb4e954b7bc0084b6f22a75d02f7842f23a56
E8M0 block	gHashTag/t27	PR #6 (v0.4.0-pre)	b211f1a863f71fd7c5e02e512efff0255ebcc51521311186e01cb9992e4464bd

Таблица 8. Сопоставление пакетов с происхождением (T7).

8.5. Манифест

Файл MANIFEST_v0.4.0-pre.json в репозитории gHashTag/t27 фиксирует все шесть значений SHA-256 пакетов, якорь версии ml_dtypes и ссылку на согласование с P3109 в едином машиночитаемом документе. Последующие потребители могут проверить целостность пакета, заново вычислив SHA-256 канонического JSON-файла и сравнив с записью в манифесте.

9. Будущая работа

9.1. Трек 2: полный набор из 84 пакетов

Шесть пакетов в этой статье охватывают форматы, наиболее немедленно релевантные промышленному ML-оборудованию. Трек 2 (целевой срок III кв. 2026) расширит охват на все 84 формата каталога, для которых доступны эталонные реализации, включая:

- NVIDIA NVFP4 (элемент S1E2M1, 16-элементный блок, блочный масштаб FP8 E4M3) — закрывая структурный разрыв, задокументированный в разделе 7.2.
- Остальные записи MLLowPrecision (варианты FP6, FP4, NF4).
- Типы Posit/Unum III через оракул libtakum.
- Варианты GoldenFloat от GF4 до GF256.

9.2. Соответствие на уровне операций

Текущий набор охватывает только уровень представления. Трек 2 добавит векторы уровня операций, согласованные с подмножеством StandardOperations.yaml стандарта P3109, начиная с округления NearestTiesToEven для сложения, умножения и FMA по всем шести текущим форматам. Это позволит командам компиляторов и оборудования проверять не

только корректность кодирования/декодирования, но и согласие их арифметических операций со стандартом на битовом уровне.

9.3. Фаззинг циклического кодирования-декодирования

Фаззер на основе свойств дополнит созданные вручную граничные векторы. Фаззер будет генерировать случайные входы FP32, применять кодирование/декодирование для каждого формата и проверять свойство циклического кодирования-декодирования и согласованность `abs_error`. Это особенно ценно для форматов со сложным поведением насыщения.

9.4. Открытые приглашения

Наиболее полезный непосредственный следующий шаг — чтобы независимый сопровождающий взял любой отдельный пакет соответствия, запустил его относительно собственной реализации и сообщил о расхождениях в виде задач (issues) на GitHub в `gHashTag/t27`. В частности:

- **ml_dtypes:** подтвердить совпадение 21/21 по BF16 и результаты 15/16 + 17/17 по FP8 относительно последнего выпуска 0.5.x.
- **Рабочая группа OCP MX:** проверить векторы элемента MXFP4 и блочного масштаба E8M0 относительно эталонной таблицы MX v1.0.
- **Редакторы IEEE P3109:** перекрёстно проверить строки Binary8p3se / Binary4p1sf в таблице 6 относительно промежуточного отчёта v3.2.x.
- **NVIDIA NVFP4:** подтвердить параметры 16-элементного блока / масштаба FP8 E4M3, использованные в разделе 7.2.
- **Rychop, libtakum:** оракулы для уровня операций Трека 2 и подкомплекта Posit/Unum III соответственно.
- **IREE, vLLM, llama.cpp, onnxruntime:** интеграторы, наиболее вероятно затрагиваемые разрывом переполнения FP8 E4M3 при межвендорных развёртываниях.

Любое найденное новое расхождение — это особенность линейки, а не отказ набора тестов.

Пакеты соответствия, схема каталога и шаблоны кодогенерации лицензированы открыто с намерением, чтобы они стали общим ресурсом сообщества для вендор-нейтральной работы с реестром численных форматов.

10. Ограничения

Настоящие каталог и набор тестов соответствия намеренно ограничены по охвату, и мы явно формулируем граничные условия, чтобы последующие пользователи могли решить, на какие утверждения полагаться.

Уровень элемента, а не уровень операций. Все шесть пакетов уровня Tier-1 проверяют поведение циклического кодирования-декодирования на уровне элемента: значение кодируется в битовый паттерн, декодируется обратно, и результат сравнивается с эталонным оракулом. Пакеты не проверяют семантику уровня операций — умножение, накопление, активацию или ядра слитного умножения-накопления (FMA). Соответствие уровня операций (Трек 2, раздел 9.2) признаётся естественным следующим слоем, но выходит за рамки настоящего препринта.

Едиственный эталонный оракул. Пакеты уровня Tier-1 используют `ml_dtypes` как эталонную реализацию, с одним хорошо задокументированным расхождением по переполнению FP8 E4M3 (раздел 7.1). Независимо реализованный оракул — например, `libtakum` для Posit/Unum III или `Rychor` для уровня операций Трека 2 — пока не подключён к стенду верификации. Расхождения, прослеживаемые к смещению одного оракула, в настоящее время нельзя исключить для форматов вне подмножества BF16 / FP8 / MXFP4 / E8M0.

Покрытие каталога асимметрично. Каталог из 84 строк (раздел 3) охватывает 13 кластеров форматов, но глубина покрытия неравномерна. IEEE 754 binary, BF16, FP8 (E4M3/E5M2), MXFP4 и семейство GoldenFloat GFN покрыты полными схемами строк, таксономией статусов утверждений и соответствующими пакетами Tier-1, где применимо. Строки Posit/Unum III, логарифмические системы счисления (LNS), NF4, BitNet, TF32 и FP6 присутствуют, но в настоящее время лишены пакетов Tier-1; их поля статуса утверждений заполнены, но ещё не проверены сквозным образом относительно оракула. Это сделано намеренно — каталог в первую очередь реестр, а во вторую — стенд верификации — но это означает, что отсутствие пакета соответствия не следует трактовать как утверждение о качестве.

NVFP4 задокументирован, но не упакован. Разрыв В (раздел 7.2) задокументирован на структурном уровне таблицей параметров, но соответствующий пакет NVFP4 уровня Tier-1 пока не включён в этот препринт. Количественные межпакетные векторы расхождений на представительных границах квантования масштаба перечислены как первый результат Трека 2 (раздел 9.1).

Никаких утверждений об оптимальности. Каталог не утверждает, что какой-либо включённый формат является наилучшим для какой-либо последующей задачи. Сравнения касаются интерпретации в рамках спецификации, а не точности, пропускной способности, энергопотребления или качества модели. Бенчмарки, ранжирующие форматы по точности или перплексии, выходят за рамки этой работы; настоящий вклад — это вендор-нейтральный справочник, а не конкурентная оценка.

Честная `abs_error`, а не нулевая `abs_error`. Норма честности из раздела 7.3 требует, чтобы каждый вектор с ненулевой ошибкой декодирования сообщал фактическую величину. Это заставляет набор выглядеть хуже, чем набор, маскирующий переполнение-в-Inf или

потерю-значимости-в-ноль как «совпадение». Потребители, сравнивающие настоящие результаты с наборами, подавляющими такие поля, должны нормализовать сравнение перед выводами.

Конверт воспроизводимости. Все якоря SHA-256, коммиты репозитория и манифесты пакетов, приведённые в разделе 8, были проверены на момент написания. Долгосрочная воспроизводимость зависит от доступности вышестоящих источников `ml_dtypes`, спецификации OCP MX и промежуточного документа IEEE P3109; если какой-либо из них станет недоступен, стелу верификации потребуется слой зеркалирования, который в настоящее время не реализован.

Отсутствие зависимости от неопубликованных работ. Результаты этой статьи — каталог из 84 форматов, шесть пакетов соответствия, кросс-уолк к P3109 v3.2.0 и два задокументированных интерпретационных разрыва — зависят только от артефактов, процитированных в библиографии и публично доступных (репозитории с открытой лицензией, опубликованные стандарты и препринт arXiv GoldenFloat). Ни одно утверждение, таблица или теорема в этой статье не опирается на неопубликованные или находящиеся на рецензировании рукописи. Предстоящие последующие работы по смежным темам упоминаются только как направление будущей работы в разделе 9 и явно выходят за рамки настоящего изложения.

Ни одно из этих ограничений не меняет центрального утверждения статьи: каталог из 84 форматов с шестью пакетами соответствия, кросс-уолком к IEEE P3109 и двумя задокументированными интерпретационными разрывами теперь является пригодным, воспроизводимым вендор-нейтральным справочником. Они очерчивают то, чем артефакт является, в отличие от того, чем он не является.

Благодарности

Автор благодарит сопровождающих `ml_dtypes` (Google/JAX) за предоставление эталонной реализации, а рабочую группу OCP Microscaling — за публикацию спецификации MX v1.0 в открытом доступе. Эта работа была выполнена в Trinity S³AI автором (ORCID 0009-0008-4294-6159). Автор благодарит рецензентов Tenstorrent tt-metal @amahmudTT и @rtawfik01 за содержательную обратную связь по смежным веткам соответствия FP8, которая повлияла на постановку раздела 7. Никакой внешней финансовой поддержки в подготовке этого препринта не использовалось.

Список литературы

- [1] D. Vasilev, «GoldenFloat: A phi-anchored numeric format family and the identity $\phi^2 + 1/\phi^2 = 3$ », arXiv:2606.05017, 2026. <https://arxiv.org/abs/2606.05017>
- [2] C. Hunhold, «Takum arithmetic: A new paradigm for low-precision numerics», arXiv:2412.20273, 2024. <https://arxiv.org/abs/2412.20273>

- [3] C. Park, J.-H. Lim, S. Nagarakatte, «ProofWright: Towards verified floating-point arithmetic», arXiv:2511.12294v2, 2025. <https://arxiv.org/abs/2511.12294>
- [4] C. Chang, C. Park, J.-H. Lim, S. Nagarakatte, «P3109 FLoPS: A Lean 4 formalization of IEEE P3109 floating-point semantics», arXiv:2602.15965, 2026. <https://arxiv.org/abs/2602.15965>
- [5] B. Rouhani et al., «Microscaling data formats for deep learning», arXiv:2310.10537, 2023. <https://arxiv.org/abs/2310.10537>
- [6] A. Ouyang et al., «KernelBench: Can LLMs write efficient GPU kernels?», arXiv:2502.10517, 2025. <https://arxiv.org/abs/2502.10517>
- [7] NVIDIA, «Introducing NVFP4 for efficient and accurate low-precision inference», NVIDIA Technical Blog, 2025. <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>
- [8] M. Wang et al., «M²XFP: A unified mixed-precision microscaling floating-point representation for next-generation AI accelerators», arXiv:2601.19213, 2026 (to appear, ASPLOS '26). <https://arxiv.org/abs/2601.19213>
- [9] N. J. Higham, «Low-precision floating-point formats: The wild west of computer arithmetic», SIAM News, 2017. <https://www.siam.org/publications/siam-news/>
- [10] E. Carson, X. Chen, «PychoP: Emulating low-precision arithmetic in Python for ML and scientific computing», arXiv:2504.07835, 2025. <https://arxiv.org/abs/2504.07835>
- [11] C. M. Wintersteiger, «Floating-point conformance testing in industrial practice», Proc. IEEE ARITH 2025. <https://arith2025.org/proceedings/215900a157.pdf>
- [12] C. Hunhold, «libtakum: A reference C library for takum arithmetic», GitHub, 2024–2025. <https://github.com/takum-arithmetic/libtakum>
- [13] C. Hunhold, T. Quinlan, «Takum arithmetic in sparse iterative solvers: A precision-vs-storage study», arXiv:2412.20268, 2024. <https://arxiv.org/abs/2412.20268>
- [14] B. Nouné et al., «8-bit numerical formats for deep neural networks», arXiv:2206.02915, 2022. <https://arxiv.org/abs/2206.02915>
- [15] IEEE Std 754-2019, IEEE Standard for Floating-Point Arithmetic, IEEE, New York, NY, 2019. <https://standards.ieee.org/ieee/754/6210/>
- [16] Open Compute Project, OCP Microscaling Formats (MX) Specification v1.0, OCP Foundation, 2023. <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>
- [17] Google/JAX, ml_dtypes version 0.5.4: Low-precision float types for NumPy and JAX, 2024. https://github.com/jax-ml/ml_dtypes
- [18] IEEE P3109 Working Group, IEEE SA P3109 Interim Report v3.2.0: Standard for Floating-Point Arithmetic for AI, IEEE Standards Association, 2026. <https://github.com/P3109/Public>